

Notes - OCR model with MNIST

- Artificial neuron (or "neuron"): is a function $f(x) = h(w \cdot x + b)$, where $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $h: \mathbb{R} \rightarrow \mathbb{R}$, $x, w \in \mathbb{R}^n$, $b \in \mathbb{R}$, " $\cdot$ " dot product

- Activation function (of a neuron): h (in the context above)

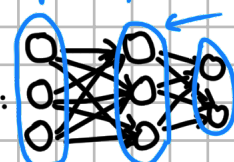
> Perceptron: a specific neuron whose act. fun. is $h(t) = \mathbb{1}_{\mathbb{R}_+}(t)$. In this case, b can be seen as a "threshold".

We can therefore think of the perceptron also as $f(x) = h(w \cdot x)$, $h(t) = \mathbb{1}_{\{t > b\}}$.

> Sigmoid function: $\sigma: \mathbb{R} \rightarrow (0,1)$, $\sigma(x) := \frac{1}{1+e^{-x}}$. This is a common activation function. A neuron whose act. fun. is the sigmoid function is called a sigmoid neuron.

"smooth" version of step function

input layer



hidden layer. There can be multiple hidden layers.

output layer.

- Neural networks: a network of neurons: example:

This is informal: for now, think like this. This could also be called a feed-forward NN, or a multi-layer perceptron (MLP), even though we don't use actual perceptrons, but neurons with continuous activation functions. Why continuous? When we make the network learn, we want to slightly change weights and biases, which would slightly change the result too (closer to what we want). With a step function like in a perceptron, a small change in w/b could cause a big change in a neuron (from 0 to 1 instantly, or vice versa), and the effects can add up, giving a big change in the result.

Creating the individual digit recognition model

0 ... white
1 ... black
other ... shade of grey

- We'll use this NN: input layer with the images are 28×28 in MNIST; each neuron will contain the greyscale value of pixel. $28 \cdot 28 = 784$ neurons, 1 hidden layer with 15 neurons, output layer with 10 neurons.
↳ we'll experiment

Learning w/ gradient descent.

